

Integrating LLMs in Java

A Practical Guide to Model Context Protocol (MCP)

Dan Vega · Spring Developer Advocate · @Brodcom

About Me

Learn more at danvega.dev



Husband & Father



Cleveland



Java Champion



Software Development 23 Years



Spring Developer Advocate



Author



O'REILLY

Fundamentals of Software Engineering

From Coder to Engineer



Nathaniel Schutta
& Dan Vega



OFFICE HOURS



<https://www.springofficehours.io>

Quick show of hands...

Who's feeling overwhelmed?

It's okay. This is a safe space.

Things we're supposed to learn right now

LLMs

Prompt Engineering

RAG

Fine-Tuning

Embeddings

Vector Databases

AI Agents

Multi-Agent Systems

MCP

A2A Protocol

Tool Calling

Function Calling

Cursor

Windsurf

Claude Code

GitHub Copilot

Vibe Coding

Agentic IDEs

Context Windows

Context Engineering

Tokens

Memory Systems

Guardrails

RLHF

GPT-5

Claude

Gemini

Llama

Mistral

Grok

DeepSeek

Qwen

Agentic Workflows

Prompt Chaining

Structured Output

Sampling

Evals

Observability

LoRA

Hallucination Mitigation

Constitutional AI

Chain of Thought

Computer Use

MoE

...and that's just this week

The good news?

**You don't need to
learn all of this.**

You need to understand the
building blocks.

The AI Developer Stack

Your Application

The thing your users actually care about

Agents & Workflows

Orchestration, Multi-Agent, Agentic IDEs, Evals

Tools & Actions

Function Calling, Tool Use, APIs, MCP

Context & Memory

Prompts, RAG, Embeddings, Vector DBs, Context Windows

Models

GPT, Claude, Gemini, Llama, Mistral, ...

Today's focus

We're going to zoom into the **Tools & Actions** layer, specifically MCP.

MCP is a single protocol that sits between your AI models and the outside world. Master this one building block, and a huge chunk of that wall of terms starts to make sense.

Start with the layer that gives your models superpowers

AI

MCP



NOVEMBER 2024

AI

PROTOCOL

AI

MODEL CONTEXT PROTOCOL

So let's start simple.

What happens when an LLM isn't enough?

Understanding LLM limitations → Tools → MCP

“

LLMs are like super-smart interns.

Brilliant

- Draft design docs in seconds
- Write complex code
- Analyze data & summarize research
- Explain anything to anyone

Confidently Wrong

- Invent APIs that don't exist
- Fabricate citations & statistics
- "The Eiffel Tower was built in 1875"
- All with total confidence

LLM Limitations

For all the good, there are real constraints we need to address

● Hallucinations

Invents facts and API names with total confidence

● Stale Data

Knowledge frozen at training cutoff — no live info

● Bias & Safety

Can output stereotypes, toxic language, or policy violations

● Domain Gaps

Uses generic wording where niche jargon is required

● Context Window

Long threads get truncated; model forgets earlier details

● Non-Deterministic

Same prompt, different answer — flaky tests, review churn

● Privacy & Security

Proprietary data could leave your trusted boundary

● Cost & Latency

High-token chains drain budgets and slow UX

● Weak Reasoning

Multi-step calculations and logical deductions fail

So how do we fix this? We have a Swiss-army lineup of solutions.

Taming LLM Limitations

Four levers: start simple, escalate as needed

1



Prompt Guarding

Encode rules that constrain behavior like tone, honesty, refusal policy. Think of it as terms of employment for our smart intern.

2



Prompt Stuffing / RAG

Inject fresh, task-specific context so the model quotes facts instead of guessing. Shove the answer into the context window.

3



Tools / Function Calling

Let the model invoke code or APIs for real-time data, calculations, and business logic. In Spring AI, that's one annotation.

4



MCP (Resources + Prompts + Tools)

Package those tools as reusable, versioned endpoints every client can share. Build once, use everywhere.

Start with prompt rules → graduate to stuffing → escalate to tools → wrap best tools in MCP

Two Categories of Tool Use

Information Retrieval

Augment the model's knowledge with real-time external data

- What's tomorrow's date?
- Current weather forecast
- Stock price for AAPL
- What are Dan Vega's latest YouTube Videos?
- What talks are at this conference?

Taking Action

Automate tasks that would otherwise need human intervention

- Send an email
- Create a database record
- Submit a form or PR
- Trigger a CI/CD workflow
- Post a message to Slack

Tools give LLMs hands and feet...

Tools in Spring AI

One annotation is all it takes

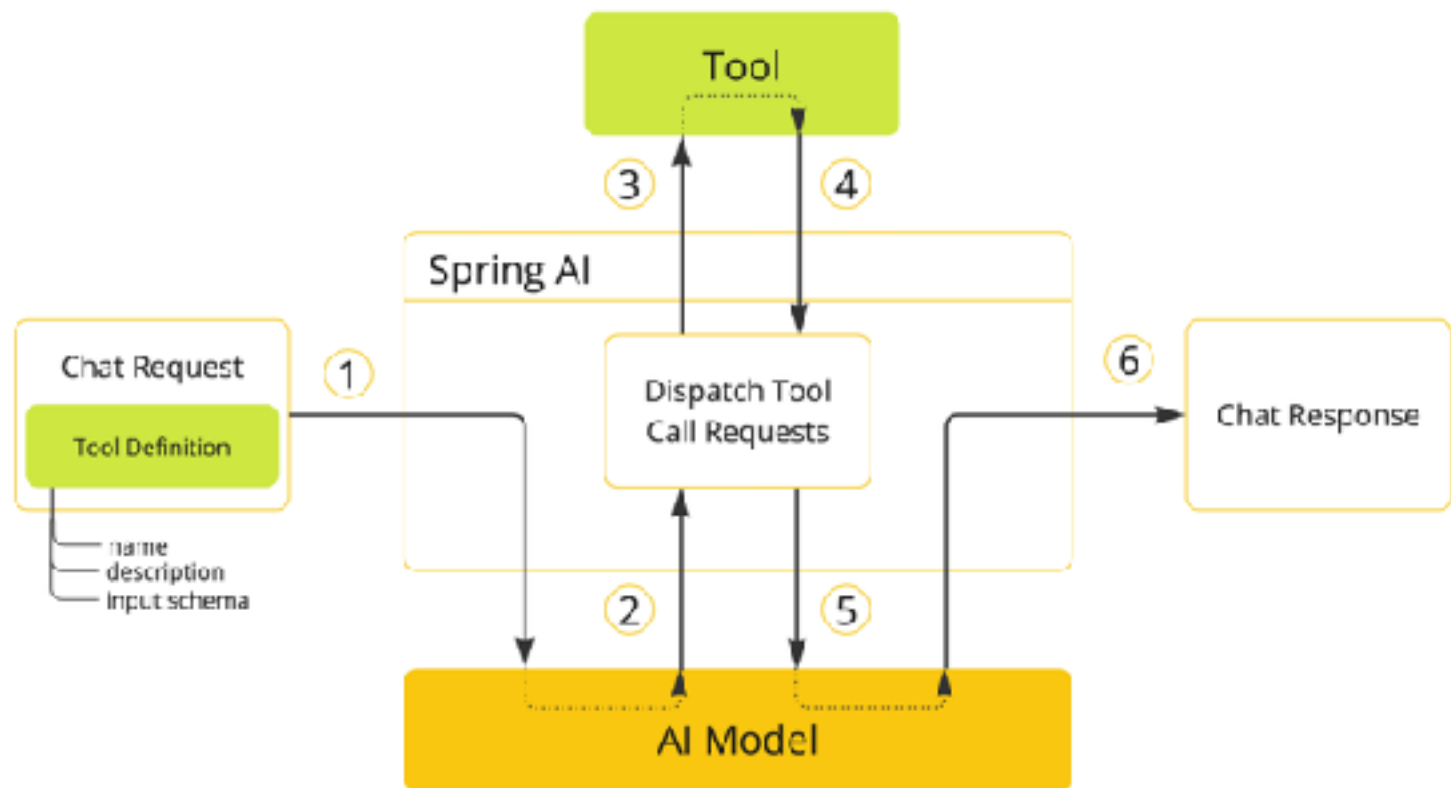
```
public class DateTimeTools {  
  
    @Tool(description = "Get the current date & time")  
    String getCurrentDateTime() {  
        return LocalDateTime.now()  
            .atZone(LocaleContextHolder  
                .getTimeZone().toZoneId())  
            .toString();  
    }  
}
```

Name + Description

The @Tool annotation registers this method. The description tells the model when to use it.

Model Decides

You don't call the tool — the AI model reads the description and autonomously decides when this tool is needed.



Tokens & LLM Pricing

Understanding the currency of AI

Producing Pricing

This is pricing for the the products that sit on top of the LLMs

Free Intelligence for everyday tasks \$0 / month Get Free ↗ ✓ Limited access to flagship model GPT-5.2 ✓ Limited messages and uploads ✓ Limited and slower image generation ✓ Limited deep research ✓ Limited memory and context Have an existing plan? See billing info	Go Keep chatting with expanded access \$8 / month Get Go ↗ ↗ Everything in Free and: ✓ More access to our flagship model GPT-5.2 ✓ More messages ✓ More uploads ✓ More image creation ✓ Larger memory This plan may include ads. Learn more	Plus Go more with advanced intelligence \$20 / month Get Plus ↗ ↗ Everything in Go and: ✓ Advanced reasoning models ✓ Expanded messages and uploads ✓ Expanded and faster image creation ✓ Expanded deep research and agent mode ✓ Expanded memory and context ✓ Projects, tasks, and custom GPTs ✓ Codex agent and Sora video generation ✓ Early access to new features	Pro Full access to the best of ChatGPT \$200 / month Get Pro ↗ ↗ Everything in Plus and: ✓ Pro reasoning with GPT-5.2 Pro ✓ Unlimited GPT-5.2 and file uploads ✓ Unlimited and faster image creation ✓ Maximum deep research and agent mode ✓ Maximum memory and context ✓ Expanded projects, tasks, and custom GPTs ✓ Expanded access to Sora video generation ✓ Expanded, priority-speed Codex agent ✓ Research preview of new features Unlimited subject to abuse guardrails. Learn more
---	---	---	---

API Pricing

API Pricing is based on Tokens

GPT-5.2

The best model for coding and agentic tasks across industries

Price

Input:

\$1.750 / 1M tokens

Cached input:

\$0.175 / 1M tokens

Output:

\$14.000 / 1M tokens

GPT-5.2 pro

The smartest and most precise model

Price

Input:

\$21.00 / 1M tokens

Cached input:

-

Output:

\$168.00 / 1M tokens

GPT-5 mini

A faster, cheaper version of GPT-5 for well-defined tasks

Price

Input:

\$0.250 / 1M tokens

Cached input:

\$0.025 / 1M tokens

Output:

\$2.000 / 1M tokens

What is a Token?

~3/4

of a word per token

100 tokens $\approx$ 75 words

1 token $\approx$ 4 characters

Why it matters

Everything going to and from the model is measured in tokens. More tokens = more cost. Tools add tokens too.

Tokenizer Example

Tell me an interesting fact about Java

7 tokens · 38 characters

[60751, 668, 448, 9559, 2840, 1078, 13114]

Context Window

← Context Window Size (e.g., 200K tokens) →

System

User

Assistant

Tools

TOKENS

LLM Pricing Landscape

Per 1M tokens · Prices as of mid-2025

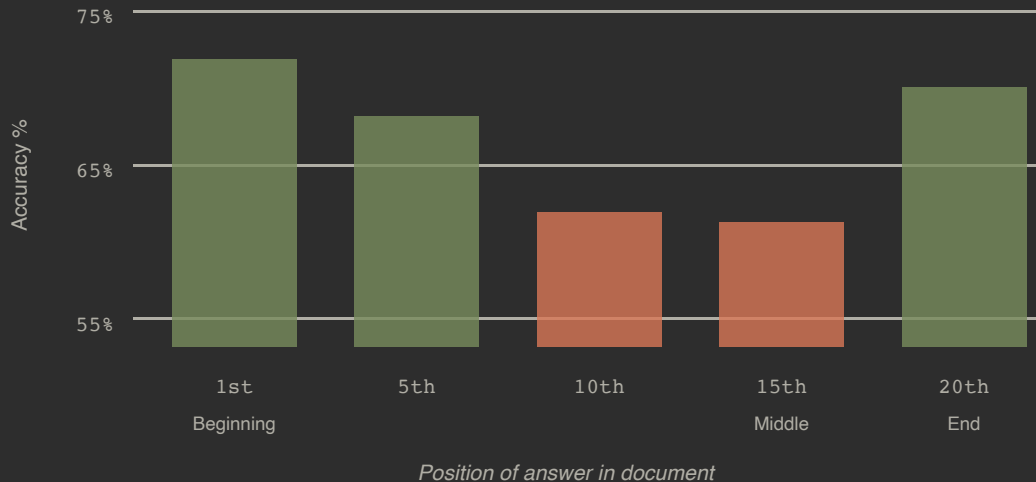
Model	Context	Input	Output	Notes
GPT-5 (OpenAI)	~400K	\$1.25	\$10.00	Cached: \$0.125
GPT-5 Mini	~400K	\$0.25	\$2.00	Cached: \$0.025
GPT-5 Nano	~400K	\$0.05	\$0.40	Cached: \$0.005
Claude Sonnet 4	200K	\$3.00	\$15.00	
Claude Opus 4.1	200K	\$15.00	\$75.00	32K output
Gemini 2.5 Flash-Lite	1M	\$0.10	\$0.40	
Gemini 2.5 Flash	1M	\$0.30	\$1.25	
Gemini 2.5 Pro	1M	\$1.25	\$10.00	>200K: \$2.50/\$15
Grok 3 (xAI)	131K	\$3.00	\$15.00	Cached: \$0.75

Key insight: A single tool call can add 500-2,000 tokens of overhead. With 10 tools available, that's up to 20K tokens before the user even asks a question.

Context Rot

Bigger context windows don't mean better answers

Accuracy vs. Position in Context



Lost in the Middle

Liu et al., 2023

LLMs are better at using info at the beginning or end of context. Performance degrades significantly in the middle.

Context Length Hurts

Du et al., 2025

Even with perfect retrieval, performance still degrades 13-85% as input length increases within claimed limits.

Stuffing more context isn't always the answer. This is why tools and MCP matter.

The Hidden Cost of Tools

Every tool you register eats context, even when it's not used

200K Token Context Window

System
Prompt

Tool Definitions (10 tools × ~500 tokens each)

Chat
History

Available for
user query + response

Selection Accuracy

More tools means more chances for the model to pick the wrong one. Beyond ~20 tools, decision quality drops significantly.

Context Budget

Each tool definition costs 200-2,000 tokens. Register 50 tools and you've burned 25-100K tokens before the conversation starts.

Not Reusable

Traditional tools are wired into one application. Want them in Slack, IntelliJ, and a CLI? Rewrite the integration 3 times.

This is the exact problem MCP solves... standardized, reusable, shareable tool endpoints

Model Context Protocol

Exploring the building blocks of MCP

“

MCP is a standardized protocol for giving LLMs structured access to tools, data, and systems.

The Problem MCP Solves

Build once and use everywhere instead of rebuilding integrations for every client.

Without MCP

- ✗ Rewrite tool integrations per app
- ✗ Each client has its own bugs
- ✗ Separate maintenance burden
- ✗ Provider lock-in

With MCP

- ✓ Build a server once, any client connects
- ✓ Standardized protocol, fewer bugs
- ✓ One codebase to maintain
- ✓ Model and provider agnostic

Benefits of MCP



Modularity

Keep AI apps light while deeply environment-aware



Reusability

One server serves Claude, Cursor, IntelliJ, and more



Language Agnostic

Works across Java, Python, TypeScript, and more



Fine-Grained Control

Decide what data and tools your AI can access



Privacy & Security

Keep sensitive data local, you control visibility

Key Insight: MCP lets you package tools, context, and prompts as reusable, versioned endpoints that any AI client can discover and use.

MCP Server vs. Traditional API

Both expose functionality but they serve fundamentally different consumers.

Traditional API

Consumer: Software systems & developers

Discovery: Requires docs and manual integration

Schema: REST, GraphQL, gRPC — varies

Decision: Developer decides which endpoint to call

Context: Not designed for AI context windows

MCP Server

Consumer: LLMs and AI applications

Discovery: Auto-discovered by AI clients

Schema: JSON-RPC 2.0 — always consistent

Decision: Model autonomously picks the right tool

Context: Purpose-built for AI context windows

“

API's are for systems
MCP's are for LLMS

MCP in Action

Build one MCP server → every AI client gets the same capabilities

Your MCP Server

Java / Spring AI



Claude Desktop

Cursor / IDEs

Spring Boot App

CLI Tools

Zero code changes. Write your MCP server once in Java. Every MCP-compatible client discovers and uses it automatically.

Why Java Developers Should Care

Java and Spring are first-class citizens in the MCP ecosystem



Official Java SDK

Spring donated the official MCP SDK for Java. This isn't a third-party wrapper, it's the reference implementation.



Spring AI Integration

Spring AI has first-class MCP support. Use @McpTool, auto-configuration, and the Spring programming model you already know.



Your Skills Transfer

Dependency injection, testing, security, observability... everything you know about Spring applies directly to MCP servers.



Enterprise Ready

OAuth2, Spring Security, Spring Authorization Server... production-grade security from day one.

PRIMITIVES

The Building Blocks of MCP Servers

MCP Server Primitives

Tools

Model-controlled: **Claude** decides when to call these. Results are used by **Claude**

Used for:

- Giving additional functionality to Claude

Resources

App-controlled: **Our app** decides when to call these. Results are used primarily by **our app**.

Used for:

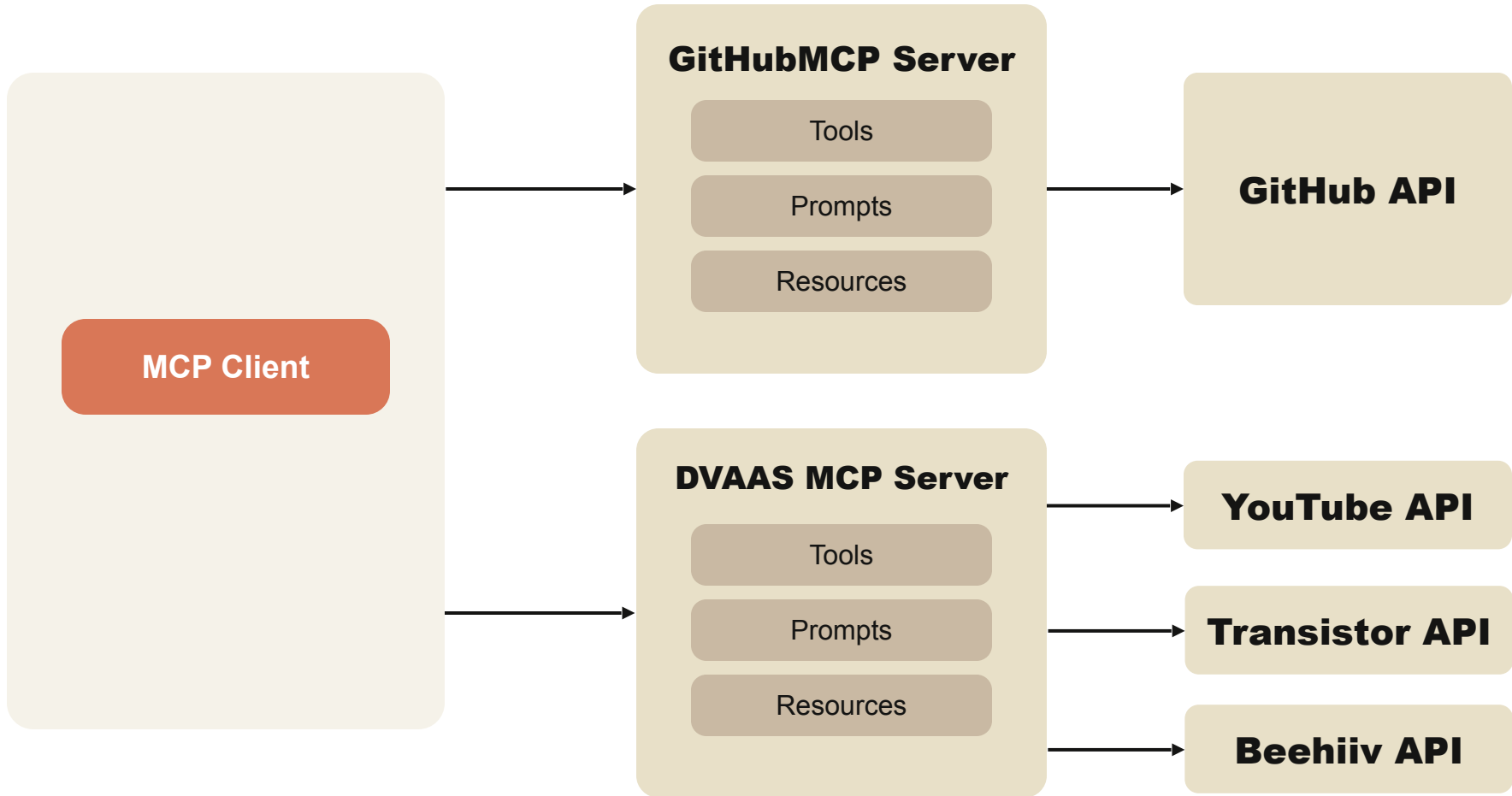
- Getting data into our app
- Adding context to messages

Prompts

User-controlled: **The user** decides when to use these.

Used for:

- Workflows to run based on user input, like a slash command, button click, or menu option



MCP Primitives: Tools

WHAT

Executable functions that AI applications can invoke to perform actions (e.g., file operations, API calls, database queries)

WHEN

AI needs to take action beyond just generating text — when it needs to DO something in the real world

EXAMPLES

read_file() Read contents of a file

write_file() Create or modify files

execute_sql() Run database queries

send_email() Send messages via email API

git_commit() Commit changes to repository

slack_post() Send messages to Slack channels

web_search() Search the internet

calculate() Perform mathematical operations

Tools are model-driven — the AI decides when to call them. Think function calling, standardized through MCP.

MCP Primitives: Resources

WHAT

Data sources that provide contextual information to AI applications (e.g., file contents, database records, API responses)

WHEN

AI needs to understand or reference existing information before responding or taking action

EXAMPLES

file://project/README.md Documentation and project files

db://users/profile/123 User records and data

git://repo/commit/history Version control information

slack://channel/messages Chat history and conversations

calendar://events/today Schedule and meeting data

email://inbox/recent Email content and metadata

Resources are application-driven — your app decides when to load them. Think of it as giving AI access to your information universe.

MCP Primitives: Prompts

WHAT

Reusable templates that help structure interactions with language models (e.g., system prompts, few-shot examples)

WHEN

You need consistent, well-crafted prompts across different conversations or want to standardize AI behavior patterns

EXAMPLES

code_reviewer Template for reviewing PRs with specific criteria

meeting_summarizer Structured format for extracting action items

technical_writer Guidelines for creating docs in company style

bug_triager Template for categorizing and prioritizing issues

customer_support Consistent tone and approach for user interactions

data_analyst Framework for interpreting charts and metrics

Prompts are user-driven — the user decides when to use them. Think prompt engineering, but modular and shareable.

Primitives: Interaction Model

Who controls each primitive, and what does it provide?

PRIMITIVE	CONTROLLED BY	PROVIDES
Tools	Model-Driven The AI model decides when to call these	Actions Execute functions, call APIs, modify data
Resources	Application-Driven Your app decides when to load these	Context Files, database records, memory
Prompts	User-Driven The user decides when to use these	Workflows Added to the context window on demand

TRANSPORTS

The Foundation for communication between clients and servers

“

Transports in the Model Context Protocol (MCP) provide the foundation for communication between clients and servers. A transport handles the underlying mechanics of how messages are sent and received.

Transports

How messages are sent and received between MCP clients and servers. Uses JSON-RPC 2.0 as its wire format.

Standard I/O

stdio

- Building command-line tools
- Implementing local integrations
- Simple process communication
- Working with shell scripts

Server-Sent Events

SSE

- Server-to-client streaming only
- Working with restricted networks
- Implementing simple updates

Streamable HTTP

HTTP

- Building web-based integrations
- Bidirectional streaming
- Request / Response streaming
- Modern HTTP infrastructure

SECURITY

Securing MCP Servers that can scale from local dev to global

Authorization **Allows**

Private Context Sharing

Allows private context to be shared from trusted data sources. Creates a trust boundary so MCP servers can access internal documents, customer data, and proprietary systems.

Account Binding

Enable MCP server authors to bind the capabilities of a server to an account. The same server can provide different functionality based on who's using it.

Third-Party Integrations

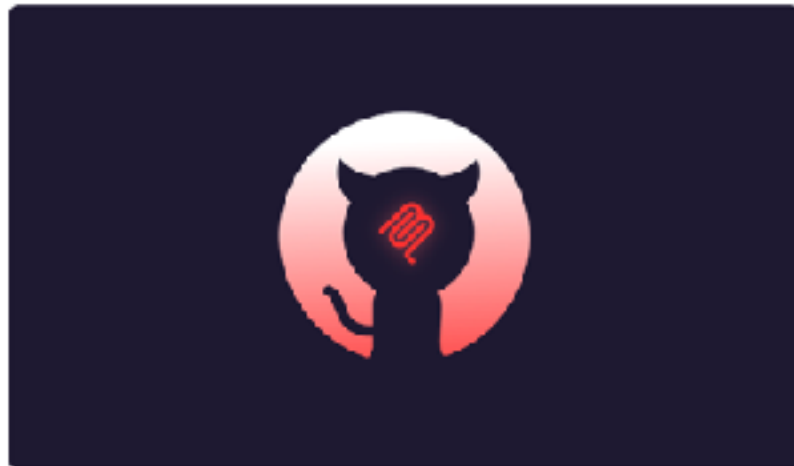
Securely connect to third-party integrations. Leverage existing OAuth providers, SAML systems, or enterprise identity providers like Salesforce, GitHub, and internal databases.

Without authorization, MCP servers are sandbox toys. With it, they become enterprise-ready tools.

2025-05-26

GitHub MCP Exploited: Accessing private repositories via MCP

We showcase a critical vulnerability with the official GitHub MCP server, allowing attackers to access private repository data. The vulnerability is among the first discovered by Invariant's security analyzer for detecting toxic agent flows.



Invariant has discovered a critical vulnerability affecting the widely-used [GitHub MCP integration](#) (14k stars on GitHub). The vulnerability allows an attacker to hijack a user's agent via a malicious GitHub Issue, and coerce it into leaking data from private repositories.

Securing MCP Servers

With Java / Spring

SECURITY CHALLENGE

While local MCP servers (stdio transport) may not need authentication, enterprise HTTP deployments require robust security and permission management.

OAuth2 INTEGRATION

New MCP spec (2025-03-26) leverages OAuth2 framework — MCP server acts as both **Resource Server** (validates tokens) and **Authorization Server** (issues tokens).

IMPLEMENTATION

1

Add Spring Security & Spring Authorization Server dependencies

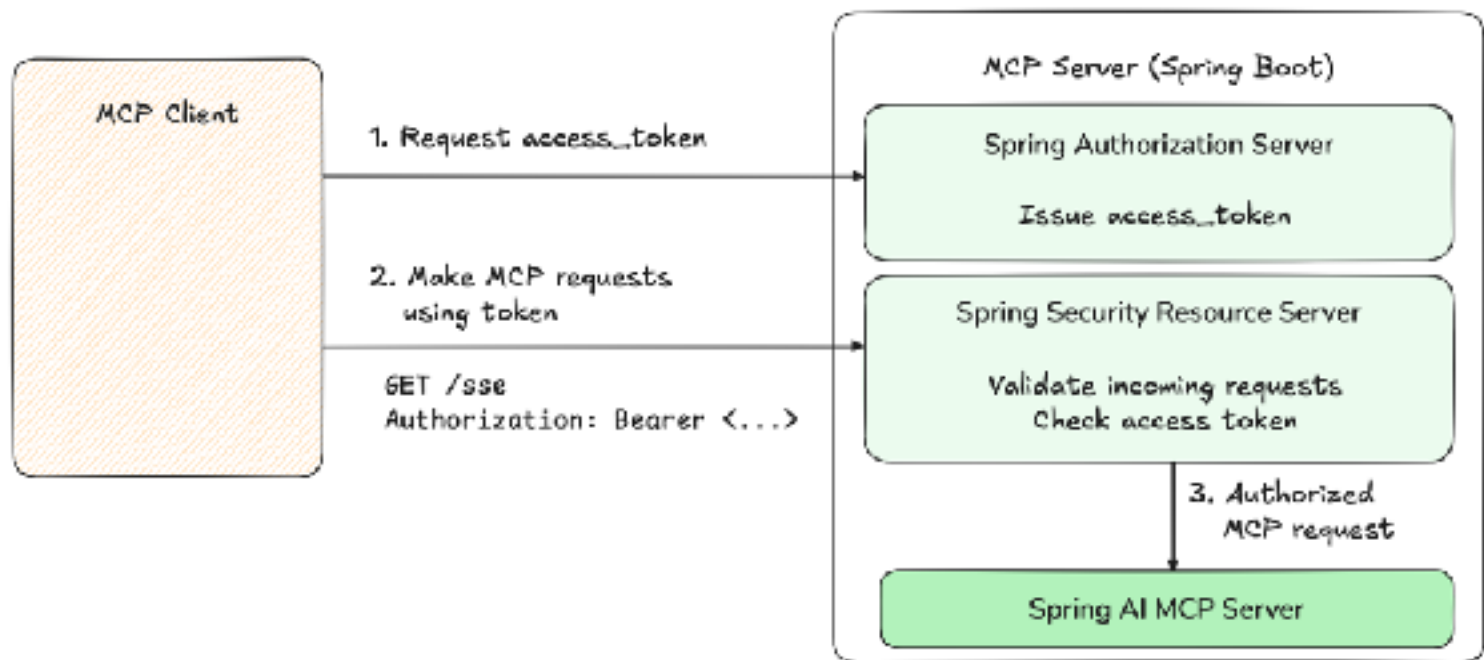
2

Configure OAuth2 client credentials in application.properties

3

Create SecurityFilterChain for authentication and token validation

Securing MCP Servers



SECURING MCP SERVERS WITH SPRING AI

<https://spring.io/blog/2025/09/30/spring-ai-mcp-server-security>

Securing MCP Servers with Spring AI

ENGINEERING | DANIEL AMMER-MOORE | SEPTEMBER 26, 2023 | 0 COMMENTS



Model Context Protocol, or MCP for short, has taken over the AI world. If you've been following our blog, you've probably read the introduction to the topic: [Connect Your AI to Everything: Spring AI's MCP Tool](#).

The security aspects of MCP have been evolving fast, and the latest version of the spec is getting more and more support from the ecosystem. To meet the needs of Spring users, we have incubated a dedicated project on GitHub: [spring-ai-community/mcp-security](#). This week, we pushed our first release, and you can now add them to your Spring AI tool-based applications. In this post, we'll explore:

- Securing MCP Servers with OAuth2
- Enabling an MCP-compatible Spring Authorization Server
- Securing MCP Servers with API Keys Instead of OAuth2

Securing MCP Servers with OAuth 2

According to the [Authorization section](#) of the MCP specification, MCP Servers exposed over HTTP must be secured with OAuth 2 access tokens. Any call to an MCP Server must have a header: `Authorization: Bearer <access.token>`, where the access token is obtained from an authorization server (think OAuth 2.0) on behalf of the user. The MCP server must also explicitly advertise the authorization servers it trusts, so MCP clients can discover them dynamically, register themselves with the auth servers, and obtain tokens. We'll discuss authorization servers later on, but for now, we'll assume you have an auth server configured and running at `<AUTH_SERVER_URL>`, and we'll hook our MCP Server to it. If you need to setup an authorization server, see [next section](#).

First, add the required dependencies to your project:

Maven:

```
<dependencies>

<!-- Spring AI MCP starter -->
<dependency>
<groupId>org.springframework.ai</groupId>
<artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<!-- MCP Security -->
<dependency>
<groupId>org.springframework.ai</groupId>
<artifactId>spring-ai-mcp-server-security</artifactId>
<version>2.0.0</version>
</dependency>
<!-- MCP Security dependencies -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-web</artifactId>
<version>6.1.10</version>
</dependency>
```



<https://www.youtube.com/@danvega>

Advanced Features



Sampling

Allows MCP servers to request LLM completions through the client, enabling agentic behaviors while the client maintains control over model access, selection, and permissions.



Elicitations

Enables servers to request additional information from users during operations using structured JSON schemas to validate responses, allowing interactive workflows while maintaining human oversight.



Completions

Provides a standardized way for servers to offer argument autocomplete suggestions for prompts and resource URIs, enabling IDE-like experiences where users receive contextual suggestions while entering values.



Progress

Supports optional progress tracking for long-running operations through notification messages, allowing either side to send updates about operation status to keep users informed.

Building MCP Servers

How to build, secure, test and deploy MCP Servers in Java & Spring

[Introduction](#)**SDKs****Concepts**[Architecture Overview](#)[Server Concepts](#)[Client Concepts](#)[Versioning](#)**Tutorials**[Using MCP](#) >[Server Development](#) >[Client Development](#) >[FAQs](#)

SDKs

[Copy page](#)

Official SDKs for building with the Model Context Protocol

Build MCP servers and clients using our official SDKs. Choose the SDK that matches your technology stack - all SDKs provide the same core functionality and full protocol support.

Available SDKs



TypeScript



Python



Go



Kotlin



Swift



Java



C#



Ruby



Rust





Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (M2)

☐ 4.0.4 (SNAPSHOT)

☒ 4.0.3

☐ 3.5.12 (SNAPSHOT)

☐ 3.5.11

Project Metadata

Group: com.danvega

Artifact: dvaas

Name: dvaas

Description: Dan Vega As A Service (dvaas)

Package name: com.danvega.dvaas

Packaging: ☒ Jar ☐ War

Configuration: ☐ Properties ☒ YAML

Java: ☒ 25 ☐ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... ⌘ - B

Spring Web WFF

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Model Context Protocol Server AI

Spring AI support for Model Context Protocol (MCP) servers.

Model Context Protocol Security (Experimental) AI

Provides security for Spring AI's MCP server and client, and for the OAuth2 Authorization Server.



GENERATE ⌘ + ↵

EXPLORE CTRL + SPACE

...

Building an MCP Server

in Java / Spring

SPRING PROGRAMMING MODEL

It's the same Spring you already know: dependency injection, annotations, auto configuration, with some additional MCP specific APIs layered on top.

MCP APIs

Tools

`@McpTool`

Executable functions the AI can invoke. Define methods, annotate them, and Spring handles the rest.

Prompts

`@McpPrompt`

Reusable templates for structuring LLM interactions. Expose prompt templates your server offers to clients.

Resources

`@McpResource`

Data sources the AI can reference. Expose files, database records, or API responses as addressable URIs.

```
@Component
public class VideoTools {

    @McpTool(name = "get-recent-videos", description = "Returns last 5 videos from Dan Vega's YouTube Channel")
    public List<Video> getRecentVideos() {
        return List.of(
            new Video("Building a Terminal UI for Spring Initializr with Java",
                "https://www.youtube.com/watch?v=J9C2MiQTIYs"),
            new Video("Spring Boot RestClient.Builder Explained (Builder Pattern)",
                "https://www.youtube.com/watch?v=aocKQ2-U3wU"),
            new Video("Spring AI Prompt Caching: Stop Wasting Money on Repeated Tokens",
                "https://www.youtube.com/watch?v=eYb7BKW4QcU"),
            new Video("Spring REST Client with Service Discovery (Eureka)",
                "https://www.youtube.com/watch?v=s9yyxyvYuq4"),
            new Video("Claude Code Tasks: Stop Babysitting Your AI Agent",
                "https://www.youtube.com/watch?v=NAWKFRaR0Sk")
        );
    }
}
```

```
@SpringBootTest
```

```
class VideoToolsTest {
```

```
    @Autowired
```

```
    VideoTools videoTools;
```

```
    @Test
```

```
    void shouldReturnFiveRecentVideos() {
```

```
        List<Video> videos = videoTools.getRecentVideos();
```

```
        assertThat(videos).hasSize(5);
```

```
    }
```

```
    @Test
```

```
    void shouldReturnVideosWithTitleAndUrl() {
```

```
        List<Video> videos = videoTools.getRecentVideos();
```

```
        assertThat(videos).allSatisfy(video → {
```

```
            assertThat(video.title()).isNotBlank();
```

```
            assertThat(video.url()).startsWith("https://www.youtube.com/watch?v=");
```

```
        });
```

```
    }
```

```
}
```

Testing Your MCP Servers

RUN YOUR SERVER

studio

Executable JAR

Package as a JAR and run as a local process. Best for IDE integrations and CLI tools.

HTTP

Run the Server

Start as a web server with Streamable HTTP transport. Best for shared/remote deployments.

TEST WITH AN MCP CLIENT

Spring MCP Client

Programmatic testing in your test suite

Claude Desktop

Interactive testing with Anthropic's desktop app

Cursor / Windsurf / Junie

Test inside your IDE's AI assistant

Any MCP Client

The protocol is open — use whatever fits

Transport Type

Streamable HTTP

URL

<http://localhost:3001/exp>

Connection Type

via Proxy

[Server Only](#) [Server File](#)[Authentication](#)[API Configuration](#)[Reconnect](#) [Disconnect](#)

Connected

[dvaas mcp server](#)
Version: 1.0.0

Tools

[List Tools](#)[Clear](#)

get-recent-videos

Returns last 5 videos from Dan Vega's YouTube Channel

get-recent-videos

Returns last 5 videos from Dan Vega's YouTube Channel

[Read-only](#) [Destructive](#) [Idempotent](#) [Open-web](#)

Tool-specific Metadata

[Add Pair](#)

No metadata pairs.

[Run Tool](#)[Copy Input](#)Tool Result: **Success**

```
{
  0: {
    title "Building a Terminal UI for Spring Initializer with Jawn"
    url "https://www.youtube.com/watch?v=29C2H20TfIs"
  }
  1: {
    title "Spring Boot RestClientBuilder Explained (Builder Pattern)"
    url "https://www.youtube.com/watch?v=oeekHQ2-UBw"
  }
  2: {
    title "Spring AI Prompt Caching: Stop Wasting Money on Repeated To
```

History

[Clear](#)

3. tools/call

2. tools/list

1. Initialize

Server Notifications

[Clear](#)

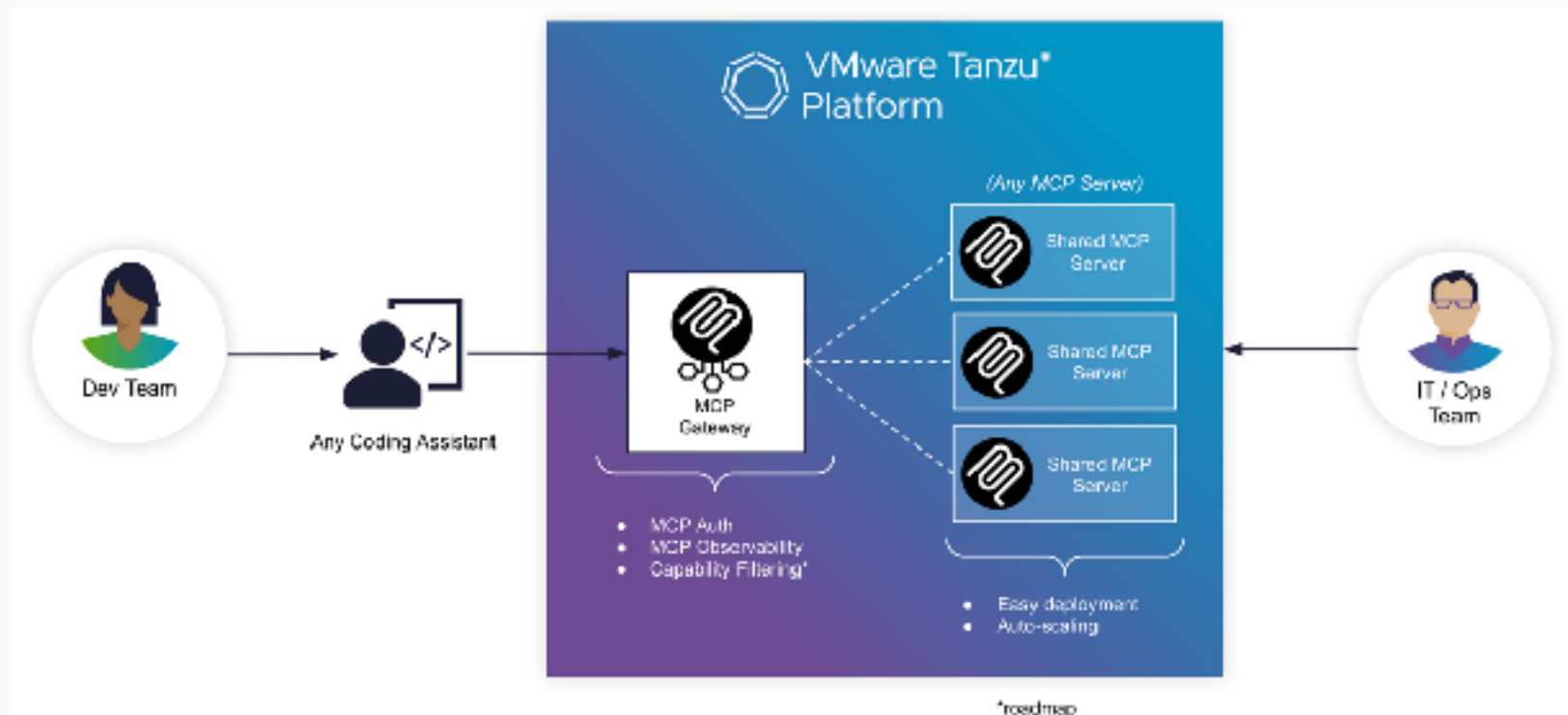
No notifications yet

“

You have an MCP Server, where and how do you deploy it for anyone to use?

Tanzu Platform Delivers Governance & Observability

Enables access and server management for cost-optimized agentic coding assistants



DEMO

Dan Vega As A Service (dvaas)

github.com/danvega/dvaas

Thank you!

danvega@gmail.com

<https://danvega.dev>

